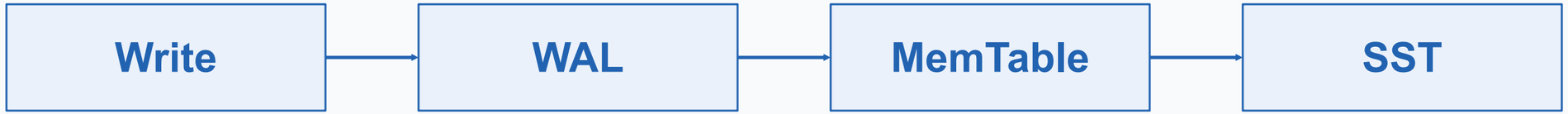
LevelDB TTL 与统计元数据的协同实现

六阶段增量开发、64 项补充测试与公开性能评测

TTL：绝对过期时间贯穿持久化链路



统计：实际内部记录随发布点统一变化

作者：待填写　学号：待填写　课程：待填写

实验目标与难点

逻辑可见性与物理记录统计采用不同口径

用户读取视图

Put(key, value, ttl)

秒级 TTL，内部统一为微秒绝对时间

Get、Snapshot、Iterator

到期边界一致，过期新版本遮挡旧值

WAL / SST / Repair

持久化与重开保留相同语义

当前存储状态

total_entries / distinct_keys / seq

覆盖 mem、imm 与 current Version

增量维护

查询直接读取计数，不扫描全库

失败不提前发布

写入和压实在成功提交后更新统计

关键难点：旧值复活、跨文件去重、失败与并发下的一致发布

统计语义契约

过期改变可见性，实际回收才改变计数

操作	total_entries	distinct_keys	seq	含义
空库	0	0	0	初始状态
Put(a, old)	1	1	1	新内部记录
Put(a, new)	2	1	2	同 key 的新版本
Delete(b)	3	2	3	删除标记也计数

示例假设历史记录尚未被压实回收

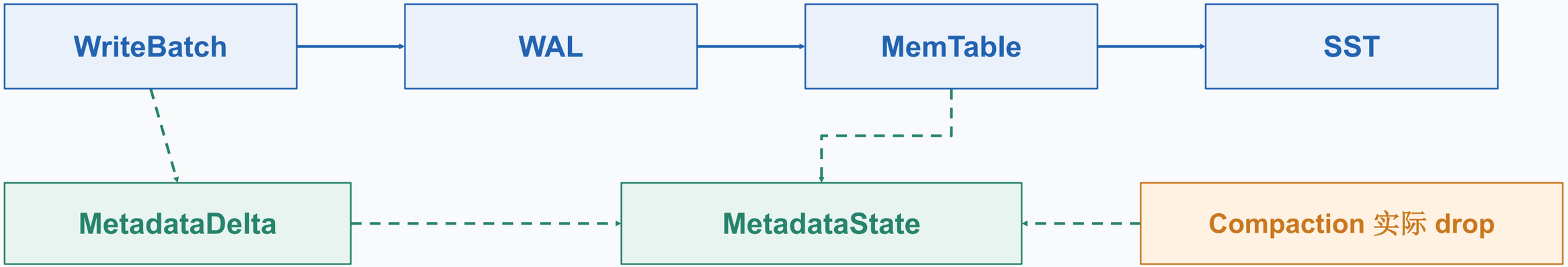
Flush 不改变全局计数，Compaction 不推进 seq

统计接口始终返回当前状态，忽略 ReadOptions::snapshot

范围仅为 mem + imm + current Version，排除已过时文件

总体架构与六阶段路线

原有数据路径上增加 TTL 语义与统计事务



重建输入：current SST 引用摘要 + WAL 恢复后存活的 mem / imm

阶段	主要内容	阶段	主要内容
01	编码、写入与 WAL 恢复	04	写入统计与原子发布
02	点查与快照时间	05	文件摘要与 MANIFEST
03	双向迭代器固定视图	06	安全回收与启动重建

TTL 编码选择

持久化绝对过期时间，保持原有内部键排序

internal key

user key	sequence：高 56 位	type：低 8 位

TTL stored value

expiration_micros：fixed64，8 B	user value：原始二进制字节

type = 0	type = 1	type = 2
deletion	value	valueWithTTL

sequence 表示版本顺序，fixed64 使用小端编码。时间字段位于 value 头部。

TTL 复用标准写入链路

WAL 重放保留原始 deadline，重启不会续命

```
uint64_t expiration_micros;
Status s = ComputeExpirationTime(
    env_, ttl, &expiration_micros);
if (!s.ok()) return s;

WriteBatch batch;
WriteBatchInternal::PutWithTTL(
    &batch, key, value, expiration_micros);
return Write(options, &batch);
```

01 计算并检查 **deadline**

02 编入 **WriteBatch**

03 复用 **Write / WAL**

04 解析后插入 **MemTable**

恢复：WAL 原字节、WriteBatch、Iterate、MemTable

兼容旧 Handler：默认 PutWithTTL 转调 Put；存储 Handler 覆盖回调以保留 deadline

边界检查与测试

零 TTL 仍写入，溢出拒绝写入

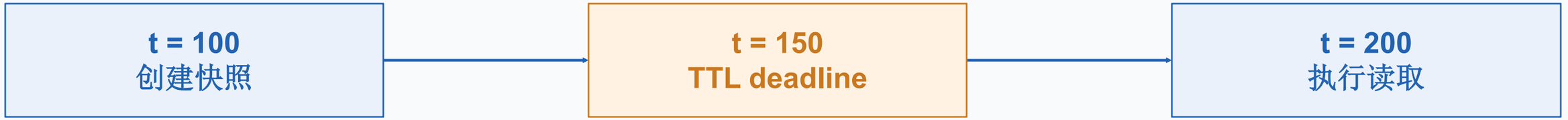
边界	处理与验证
秒换算微秒溢出	乘法前校验，返回 InvalidArgument
当前时间加 TTL 溢出	加法前校验，不进入写路径
零 TTL	新增内部记录并推进 seq，读取时立即到期
二进制值与混合批次	保留零字节，保留类型与序列顺序
WAL / Flush / 重开	恢复原始 deadline，不重新计算寿命

可控时钟验证精确边界，损坏记录拒绝解析

读取视图是二元组

每个读取视图同时固定 sequence 与 read time

读取方式	版本边界	时间边界
普通 Get	入口取 LastSequence	入口采样一次 NowMicros
Snapshot Get	快照创建时保存	快照创建时保存



示意：t = 200 的普通读不可见，t = 100 创建的快照仍可见

mem、imm 与 SST 共用同一组读取参数。边界规则：read_time >= deadline 时过期。

过期记录终止旧版本查找

过期新版本必须遮挡更旧版本

L0：a = new
TTL 已过期

→

结果：NotFound
结束查找

L2：a = old
普通旧值仍在

此旧值不得返回

MemTable::Get 返回值	含义
false	本数据源未命中，可以继续查找
true	已有结论：值、删除、过期或错误

跨数据源与时间采样测试

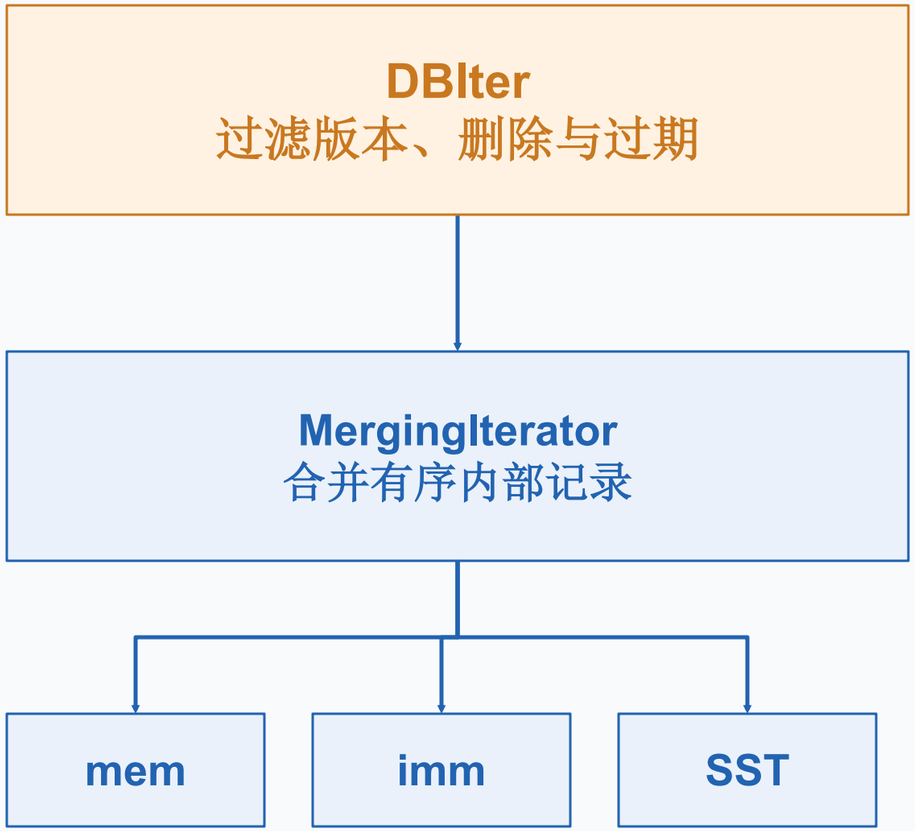
同一 TTL 规则贯穿内存、SST 与缓存

覆盖路径	主要断言
MemTable / SST	到期边界一致，二进制值完整
SST 与缓存重复读取	缓存命中仍判断原 deadline
过期 mem / imm 遮挡旧 SST	返回 NotFound，避免旧值复活
过期新 SST 遮挡旧 SST	不能继续向更老文件返回旧值
Snapshot 跨 Flush	同时固定版本与读取时间
时钟采样次数	普通 Get 一次，Snapshot Get 零次

db/ttl_internal_test.cc 采用 fake Env 控制时钟，完整测试名保留在备注

Iterator 的固定视图

创建时确定版本和时间，遍历不再采样时钟




```
if (IsExpired(expiration_micros,
              read_time_micros_)) {
  SaveKey(ikey.user_key, skip);
  skipping = true;
  break;
}
```

正向先遇到新版本

最新可见版本过期后，跳过该 key 的全部旧版本

不能只跳过当前一条

否则下次迭代可能返回同 key 的旧值

双向状态机与模型测试

反向遇到更新的过期版本时，必须清除旧候选

遍历方向	同 key 的遇见顺序	过期处理
正向	新版本在前	跳过该 key 的旧版本
反向	旧版本在前	清除已保存的候选键和值

9 项 Iterator 专项测试

Seek、Next、Prev 与方向切换

mem、imm、SST、重开与压实后视图一致

创建即固定视图，损坏错误保持可见

空库或全部过期属于正常结束

随机操作与独立最新版本模型比对

减少手写样例遗漏状态转移的风险

统计抽象的职责分工

长期状态、事务增量、局部构建与结果分离

抽象	范围	职责
MetadataState	数据库级	total 与逐 key 引用计数
MetadataDelta	一次写入 / 压实	带符号的记录数与逐 key 变化
SourceMetadata	单 MemTable / SST	total_entries、distinct_keys
SourceMetadataBuilder	有序扫描过程	摘要、逐 key 引用、has_ttl
Stats	查询输出	total_entries、distinct_keys、seq

mem = [a,b]，SST = [a,a,c]，全局引用 = {a:3,b:1,c:1}

total = 5，distinct = 3。键的等价关系由配置 Comparator 决定。

整批收集与一次发布

统计与 seq 在同一个锁内发布点对齐



解析失败：拒绝整个批次

I/O 阶段：查询仍见旧完整状态

可观察状态：旧完整批次，或新完整批次

WAL append / Sync 出错不发布，并记录后台错误阻止后续写入

O(1) 查询与增量校验

查询复制三个字段，仍可能等待 mutex

```
MutexLock lock(&mutex_);
*stats = metadata_.Snapshot(
    versions_->LastSequence());
return Status::OK();
```

total_entries

读取 total_entries_

distinct_keys

读取 key_references_.size()

seq

读取当前 LastSequence

查询本体 O(1)，增量更新需要有序 map 查找

Apply 先校验整批溢出与下溢，再修改状态

引用归零才删 key；统计始终返回当前状态

并发与失败注入测试

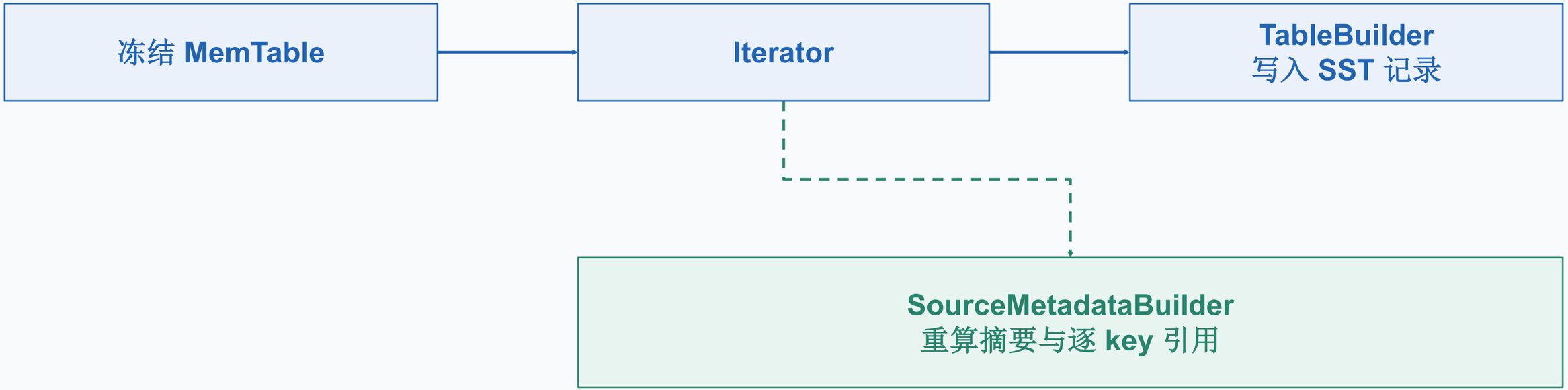
发布点之前，查询只能观察旧完整状态

测试场景	核心断言
普通 Put / TTL / Delete	每项贡献一条内部记录
Comparator 等价键	全局共享同一引用计数
非法 delta / 损坏 batch	不产生部分状态，也不提前写入
WAL append / Sync 失败	不发布，阻止后续写入
WAL 阻塞期间查询	只见上一完整批次
并发批次与统计查询	始终不暴露半批计数或混合 seq

对应测试位于 db/metadata_stats_test.cc，完整函数名列在备注

MemTable 到 SST 的局部统计

BuildTable 按实际写出记录重算摘要



WriteLevel0Table 对比 mem 与 SST 的 total / distinct

Flush 不重复更新全局计数。FileMetaData 通过 MANIFEST 持久化摘要。

逐 key 引用摘要

跨文件 distinct 无法直接相加

文件	用户 key	局部 distinct
SST A	{a,b}	2
SST B	{a,c}	2
全局去重	{a,b,c}	3

2 + 2 ≠ 3

需要逐 key 引用信息

有序输入	total / distinct	引用摘要
a, a, b, c, c	5 / 3	(a,2), (b,1), (c,2)

代价：增加 MANIFEST 体积、逐 key 内存及恢复时的摘要解析工作

MANIFEST 格式演进与兼容

unknown 与已知零分开，旧格式首次扫描升级

tag	持久化内容	恢复策略
7	原始 NewFile，无统计	unknown，首次扫描升级
10	total / distinct	缺逐 key 引用，首次扫描升级
11	摘要 + has_ttl + 逐 key 引用	直接合并引用摘要恢复

逐条重置 FileMetaData，避免新旧记录串状态

验证引用次数、合计值与 Comparator 键顺序

文件移层、MANIFEST 重写、Repair 保留或重建摘要

落盘、重写与 Repair 测试

独立扫描实际 SST 校验局部摘要

生命周期	验证内容
构建 / MemTable	有序分组、任意插入顺序、Comparator 等价
Flush	摘要持久化，全局计数不变
多次重开 / MANIFEST 重写	文件元数据持续保留
WAL 恢复 / Compaction 分裂输出	生成的每个 SST 分别重算摘要
Repair / 旧格式升级	修复后持久化，旧格式只升级一次
4 项新增 VersionEdit 测试	混合格式、known zero、截断、非法引用

独立数据校验避免用同一套计数结果验证自身

TTL 回收的安全条件

存在活跃 Snapshot 时，保守关闭 TTL 专项 GC

条件	保护目的
固定 compaction_time	本次压实使用同一时间边界
无活跃 Snapshot	避免回收快照仍可见的 TTL 记录
sequence <= smallest_snapshot	遵守版本保护边界
确实已过期	到期记录才参与 TTL drop
IsBaseLevelForKey	更深层无旧值，防止旧值复活

实际 drop 后才收集负 delta；常规多版本压实仍遵守快照边界

旧迭代器用引用保留旧数据源，当前统计只覆盖 current Version

Version 安装与负增量发布

新 Version 成功安装后，才扣除实际 drop 的记录




```
Status s = versions_->LogAndApply(
    compact->compaction->edit(), &mutex_);
if (s.ok()) {
  s = metadata_.Apply(
      compact->metadata_delta);
}
```

示意：total = 100

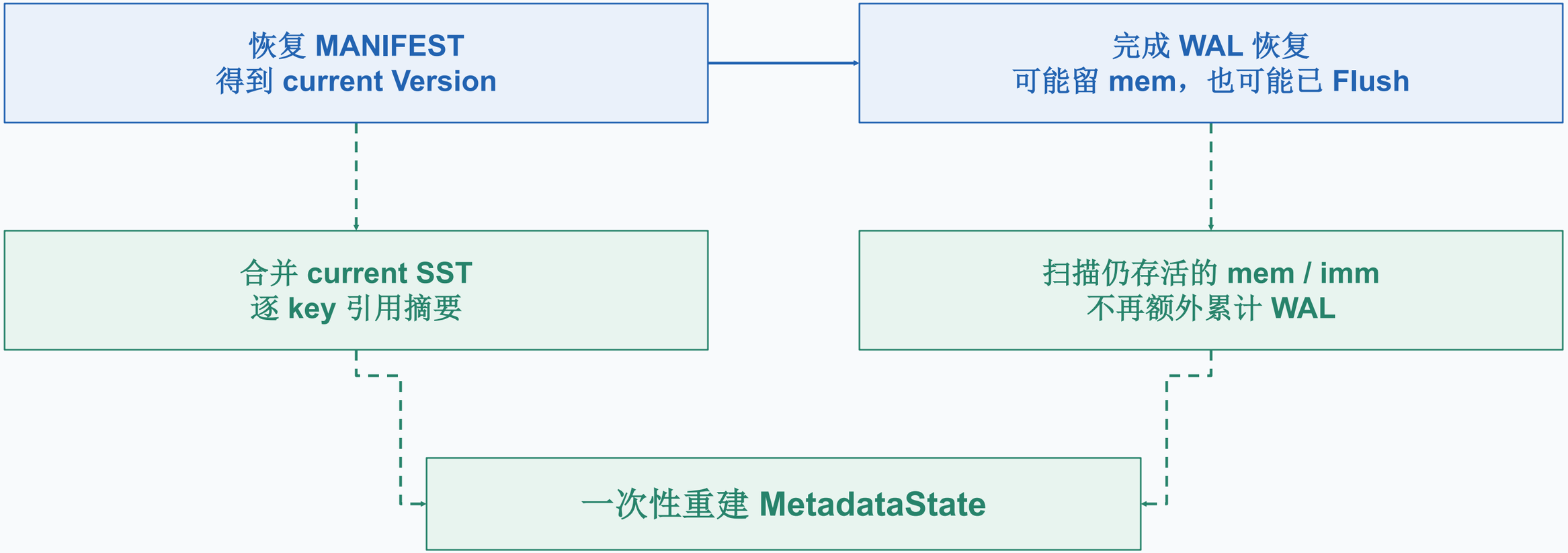
安装成功：70

安装失败：100

示意 drop = 30，seq 不变。TTL 回收避免仅做 trivial move，并支持最深层原地重写。

启动重建的当前数据集合

WAL 完成恢复后，只统计最终存活的数据源



新格式不扫描 SST 数据区，但仍需解析逐 key 引用。摘要损坏或升级失败使 Open 失败。

恢复、回收与并发联合测试

失败安装不扣数，异常退出仍可恢复

重点场景	验证断言
摘要恢复	重建全局 map，不打开 SST 数据区
回收后重开	实际删除版本数与计数变化一致
最深层 TTL GC	保护快照及 pinned Iterator
压实安装失败	暂存负 delta 不扣除
MANIFEST 写入阻塞	查询只见旧完整状态
sync 后直接退出，不析构 DB	重开仍恢复已同步写入

完整 11 项联合测试见附录 B

验证体系与新增测试

64 项新增测试覆盖边界、故障与协同状态

01 可控时钟与局部状态机

时间边界、溢出、编码、双向迭代

02 故障注入与独立校验

WAL / MANIFEST 故障，独立模型与 SST 扫描

03 并发、进程恢复与公开评测

完整发布、真实退出、原有回归与 combined

新增测试文件	数量
ttl_internal_test.cc	28
metadata_stats_test.cc	32
version_edit_test.cc	4
合计	64

VersionEdit 原有 EncodeDecode 不计入新增数，因此新增共 64 项

正确性结果汇总

公开 standard 矩阵 168 次运行全部通过

测试层级	提纲记录的结果
Debug 配置 / 构建	成功
课程 ttl_test / metadata_test	2/2 与 3/3 通过
完整 CTest	3/3 测试目标通过，0 失败，92.79 秒
combined quick	22/22 场景运行通过
combined standard matrix	22 种场景，168 次参数化 / 重复运行全通过
standard 内原生回归	C++、C、POSIX 三项 pass

CTest 的 3 项是测试目标，不是整个项目只有 3 个单元用例

来源：提纲所列 summary.json，status=pass、full_suite=true、regression=pass

性能实验方法与可复现性

Release 构建，三轮重复，逐轮配对计算影响

变量	设置
数据规模 / value	N = 10,000 / 100,000，value = 256 B
写线程 / 查询线程	写线程 1 / 8，查询线程 1
重复 / seed	每组 3 轮，基础 seed = 7774596，逐轮递增
TTL 过期比例	0% / 50% / 100%
命令配置	--kind combined --profile standard --matrix --jobs 2

配对对照：相同预加载 DB，分别执行仅写入与写入 + 持续统计查询

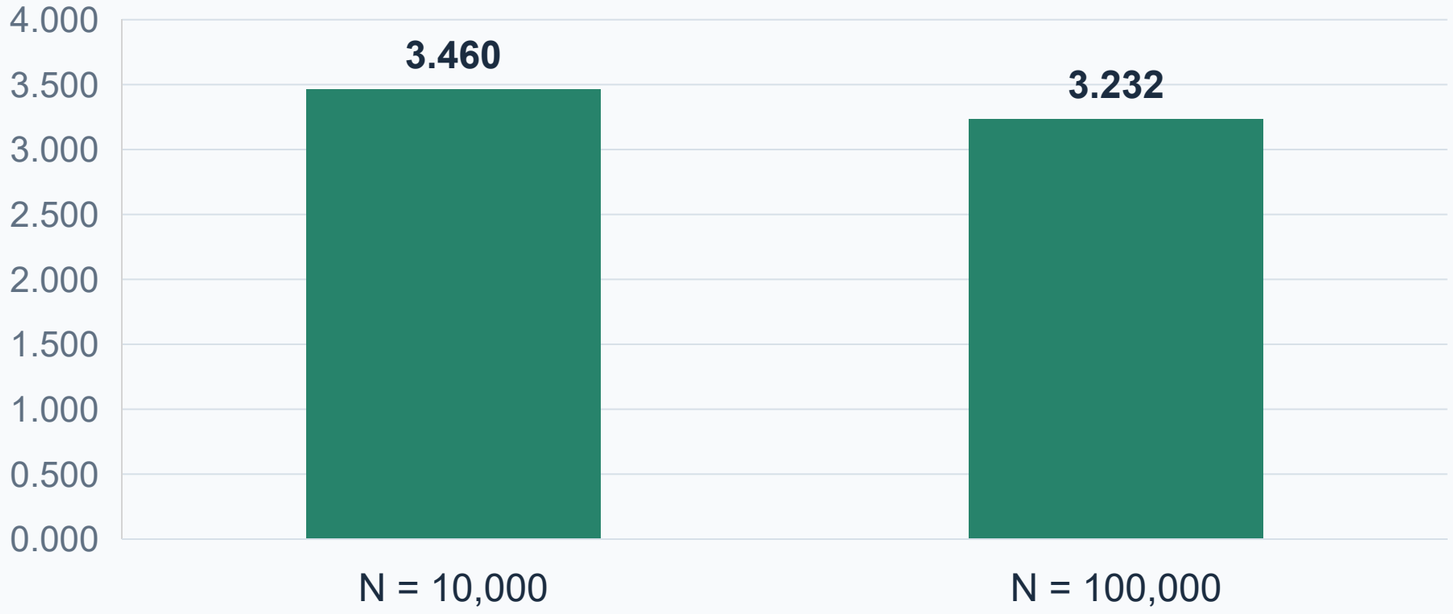
吞吐和 P99 各取中位数，范围为三轮 min–max。jobs 2 是构建并行度。

本机结果：CPU、内存、存储、OS 与编译器版本待提交前补齐

静态统计查询接近规模无关

10 倍数据量，吞吐中位数下降约 6.6%

查询吞吐（百万次 / 秒）



P99 中位数

0.046 µs

N	三轮吞吐范围
10,000	3.291–3.522
100,000	3.227–3.251

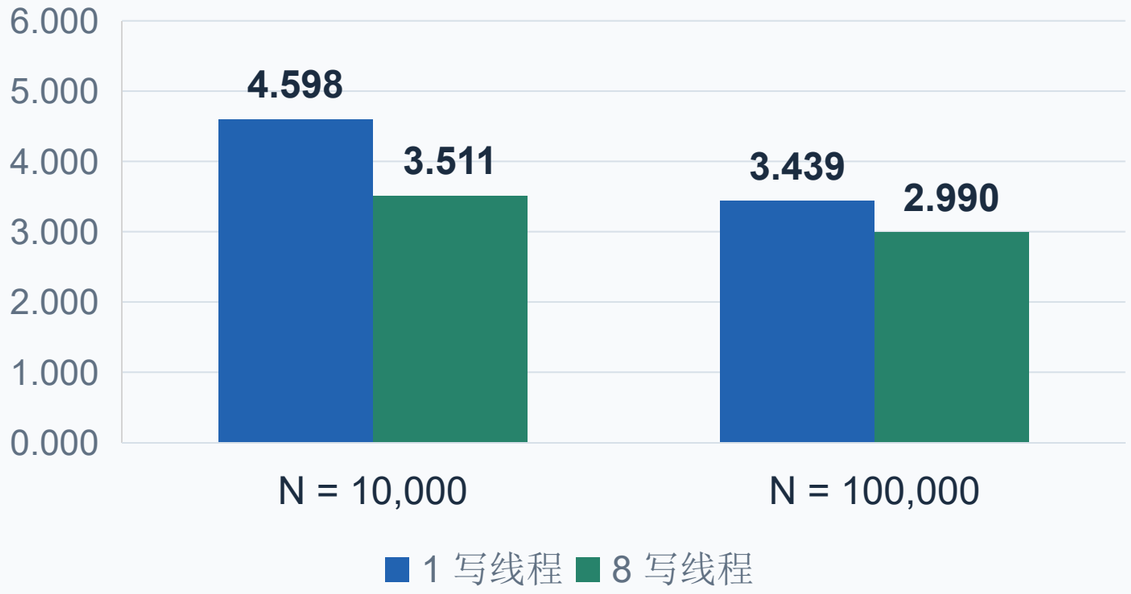
范围为三轮最小值至最大值，不是置信区间

来源：standard aggregate.json / perf_stats。吞吐口径含驱动工作，不能取倒数解释调用 P99。

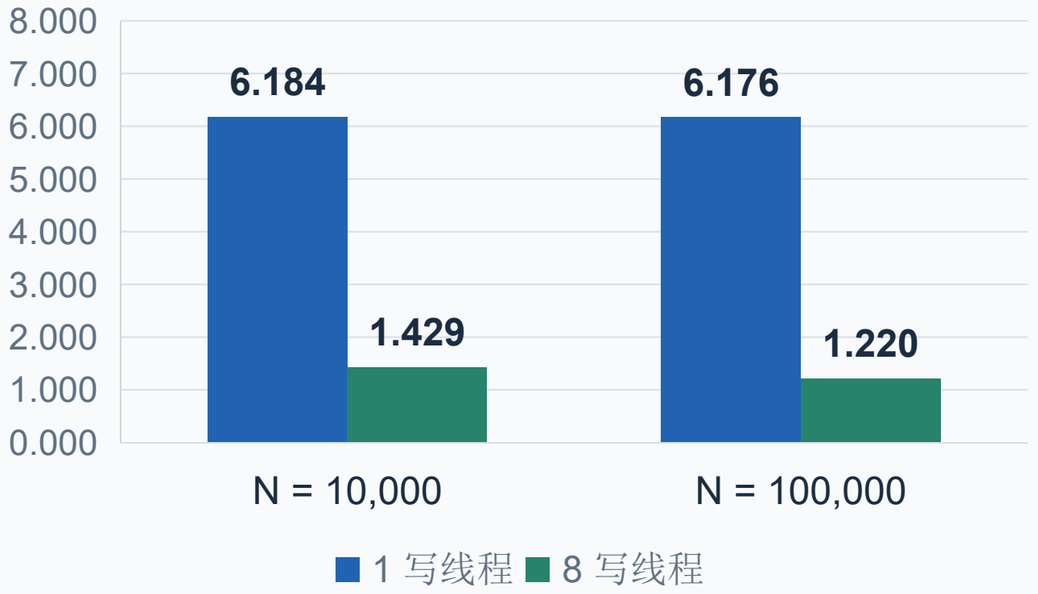
写入期间的统计查询

仍达 299 万–460 万次 / 秒，P99 受并发等待影响

查询吞吐（百万次 / 秒）



采样调用 P99（µs）



O(1) 计算仍可能等待 mutex，8 线程尾延迟较低不代表扩展性更好

来源：perf_stats_under_write。各指标为三轮中位数，完整范围见附录 C。

查询对普通写入吞吐的影响

单写线程下降约 7.5%–9.1%，多线程结果波动跨零

每轮下降率 = (1 − 有查询吞吐 / 无查询吞吐) × 100%，再取三轮中位数

N / 写线程	无查询写入 ops/s	有查询写入 ops/s	配对下降率 中位数	配对下降率 三轮范围
10,000 / 1	206,890	188,110	9.08%	0.82%–14.44%
10,000 / 8	109,579	117,644	−6.32%	−14.48%–0.82%
100,000 / 1	217,111	208,249	7.51%	3.18%–10.03%
100,000 / 8	67,475	69,050	−2.33%	−14.49%–8.14%

负中位数且范围跨零，不能据此声称查询提高写入性能

8 写线程未体现扩展收益，具体瓶颈需 profiling。数据来源：standard aggregate.json。

TTL 与统计协同性能

50% 过期比例下，单线程写入影响约 19%

N / 写线程	无查询 TTL 写入 ops/s	有查询 TTL 写入 ops/s	配对下降率 中位数	配对下降率 三轮范围
10,000 / 1	188,203	159,941	18.93%	15.02%–30.25%
10,000 / 8	66,981	68,325	−2.01%	−14.69%–19.74%
100,000 / 1	204,896	164,834	19.53%	16.05%–33.22%
100,000 / 8	69,997	61,592	11.33%	3.01%–14.37%

大规模 8 线程：影响约 11.3%

小规模 8 线程：波动不足以支持提升结论

完整评测覆盖 0% / 50% / 100%，本页只展示 50%

来源：perf_combined_stats_under_write。查询明细见附录 C。

总结与局限

可见性、实际记录和发布点共同约束正确性

完成的工作

TTL 贯通完整读写与恢复链路

统计与 **seq** 原子发布

文件摘要支持精确启动重建

安全回收、负增量与故障保护

64 项补充测试与公开评测

局限与后续

快照存在时保守延后 **TTL GC**

逐 **key map** 与摘要有空间成本

锁竞争与线程扩展性待定位

三轮结果不宣称统计显著性

尚缺原生标准 **baseline** 对照

时间决定可见性，实际记录决定统计，发布点决定一致性